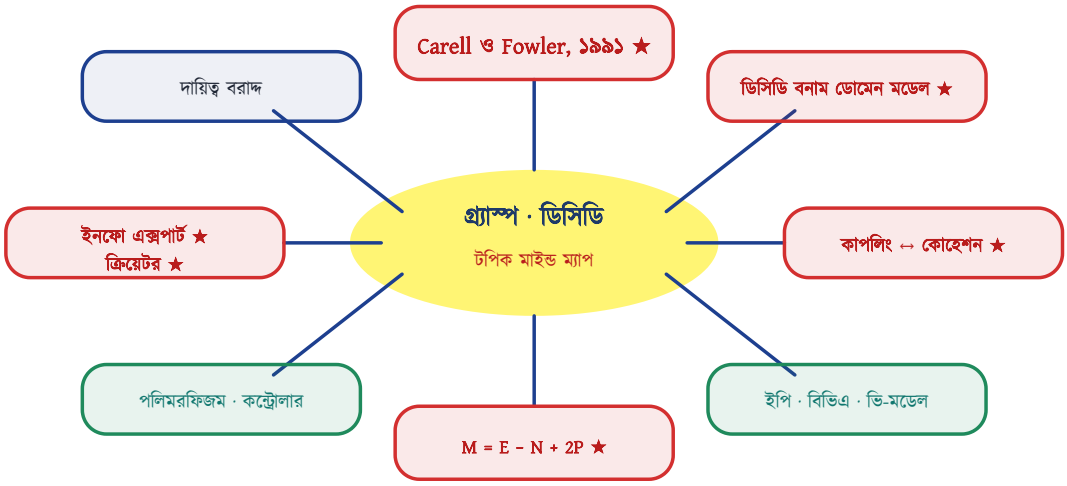
গ্র্যাস্প, ডিসিডি ও টেস্টিং কৌশল

দায়িত্ব · নয়টি নির্দেশিকা · ডিজাইন ক্লাস ডায়াগ্রাম · হোয়াইট/ব্ল্যাক বক্স · সাইক্লোম্যাটিক জটিলতা: ১১ পাতায় পরীক্ষার পূর্ণ প্রস্তুতি



↑ পুরো টপিক এক পাতায়: লাল = বেশি আসে, সবুজ = পদ্ধতি / প্রয়োগ

সিলেবাসের যে অংশ

"... **Designing Objects with responsibilities। GRASP patterns with General Principles in assigning responsibilities:** Information expert, Creator, Low Coupling and High Cohesion, **Creating design class diagrams and mapping design to codes**। Software Testing: **White Box and Black Box testing**। **Basis Path Testing**।"

হাই-ইল্ড তালিকা

১ গ্র্যাস্প = Carell ও Fowler, ১৯৯১
২ ইনফো এক্সপার্ট = তথ্য থাকা ক্লাস
৩ ক্রিয়েটর = ধারণকারী ক্লাস
৪ সবচেয়ে কম নির্ভরশীল ক্লাস জেতে
৫ ডিসিডি যোগ করে অপারেশন ও নেভিগেবিলিটি
৬ M = শর্তসংখ্যা + ১

সংকেত: **হলুদ** = মূল শব্দ · **লাল বৃত্ত** = মুখস্থ রাখার তথ্য · **বেগুনি বক্স** = মনে রাখার কৌশল · **লাল বক্স** = পরীক্ষার ফাঁদ · **সবুজ** = সমাধানকৃত উদাহরণ

সূচিপত্র

✓ গ্র্যাস্প: উৎস ও নির্দেশিকা ..... পৃ. ২
✓ ইনফো এক্সপার্ট ও ক্রিয়েটর ..... পৃ. ৩
✓ লো কাপলিং ও হাই কোহেশন ..... পৃ. ৪
✓ বাকি নির্দেশিকাগুলো ..... পৃ. ৫
✓ ডিসিডি: ডোমেন মডেল বনাম ডিজাইন ..... পৃ. ৬
✓ ডিসিডির উদাহরণ ও কোড ..... পৃ. ৭
✓ হোয়াইট বনাম ব্ল্যাক বক্স, ইপি, বিভিএ ..... পৃ. ৮
✓ কভারেজ, সাইক্লোম্যাটিক, বেসিস পাথ ..... পৃ. ৯
✓ পরীক্ষার আগের রাতের পাতা ..... পৃ. ১০
✓ অনুশীলন MCQ + লিখিত ..... পৃ. ১১

১ গ্র্যাস্প: দায়িত্ব বরাদ্দের সমস্যা

সংজ্ঞা: **গ্র্যাস্প** (GRASP) = **জি**eneral **আর**esponsibility **এ**ssignment **এস**oftware **পি**atterns: কোন অবজেক্টের ওপর কোন দায়িত্ব বসবে তা ঠিক করার **নির্দেশিকা**, যাতে ফলাফল কম কাপলড, বেশি কোহেজিভ, রক্ষণাবেক্ষণযোগ্য ও পুনর্ব্যবহারযোগ্য হয়। প্রস্তাবক **জেমস ক্যারেল ও মার্টিন ফাউলার, ১৯৯১** (*Toward Unified Design Methodologies*, *IEEE Computer*)।

দায়িত্বের দুই প্রকার

- **জানা (knowing)** — নিজের অ্যাট্রিবিউট ও সঙ্গে যুক্ত অবজেক্ট।
- **করা (doing)** — যে কাজগুলো অবজেক্টটি করে (তার অপারেশন)।

একটি **প্যাটার্ন** (Singleton, Strategy, Observer ...) হলো কাঠামোসহ একটি নামকরা সমাধান। আর একটি গ্র্যাস্প নির্দেশিকা হলো মাত্র **একটি বাক্য, যা বলে দায়িত্ব কোথায় রাখতে হবে** — এর সঙ্গে কোনো ক্লাস ডায়াগ্রাম বা কোড টেমপ্লেট থাকে না।



তিনটি প্রশ্নে সাজানো নয়টি নির্দেশিকা

নয়টি নির্দেশিকা, তিনটি প্রশ্ন

নয়টি নির্দেশিকা, প্রতিটি এক লাইনে

#	নির্দেশিকা	নীতি, এক লাইনে
১	Information Expert	তথ্য যার কাছে **আছে**, তাকেই দায়িত্ব
২	Creator	যে ক্লাস অন্যকে ধারণ বা একত্রিত করে, সে-ই তৈরি করে
৩	Low Coupling	যে ক্লাস অন্যদের ওপর **সবচেয়ে কম নির্ভরশীল**, তাকে
৪	High Cohesion	এক ক্লাসে **সর্বোচ্চ সম্পর্কযুক্ত** দায়িত্ব
৫	Polymorphism	সাবক্লাসে সুপারক্লাসের আচরণ নতুন করে লিখবেন না
৬	Pure Arbiters	যে অবজেক্ট দুই বা ততোধিকের মধ্যে **সালিশ করে**
৭	Don't Talk to Strangers	শুধু **নিজের সরাসরি সহযোগীর** কাছে জিজ্ঞাসা করুন
৮	Controller	সিস্টেম ইভেন্ট পাওয়া **প্রথম** non-UI অবজেক্ট
৯	Intermediary (Pure Fabrication)	নির্ভরতা ভাঙতে বানানো **কাল্পনিক**, ডোমেন-নিরপেক্ষ ক্লাস

গ্র্যাস্পের সঙ্গে আরও যে তিনটি আসে: Indirection — এমন একটি মধ্যবর্তী অবজেক্ট যা পরস্পর অচেনা অবজেক্টগুলোকে যুক্ত করে। Protected Variations — **অনুমিত পরিবর্তনের** প্রতিটি বিন্দুকে একটি ইন্টারফেসের ভেতরে রাখুন। Known / Unknown Dependencies — **জানা** নির্ভরতা সরাসরি উল্লেখ করুন; বদলানো **যায় এমন** নির্ভরতার সামনে ইন্টারফেস।

পরীক্ষার ফাঁদ: গ্র্যাস্প হলো **দায়িত্ব বরাদ্দের নির্দেশিকা**, প্রস্তাবক **Carell ও Fowler (১৯৯১)** — এটি প্যাটার্নের তালিকা নয়, GoF ডিজাইন প্যাটার্নও নয়। **Pure Arbiters** কখনো "Pure Arbiters / Information Expert" হিসেবে ছাপা হয়: দুটি করে গণনা করবেন না।

২ ইনফরমেশন এক্সপার্ট ও ক্রিয়েটর — উদাহরণসহ

Information Expert (ইনফরমেশন এক্সপার্ট): সেই অবজেক্টকে দায়িত্ব দিন যার কাছে কাজটি করার জন্য প্রয়োজনীয় **তথ্য** আছে। **উদাহরণ:** *Invoice*-ই *getTotal()* হিসাব করবে, কারণ LineItem-এর তালিকা কেবল Invoice-এর কাছেই আছে।

```
class LineItem {
    Product product;
    int quantity;
    double getSubTotal() {
        return product.price * quantity;
    }
}
class Invoice {
    List<LineItem> items;
    double getTotal() {
        return items.stream()
            .mapToDouble(LineItem::getSubTotal)
            .sum();
    }
}
```

এই উত্তরটি কেন সঠিক

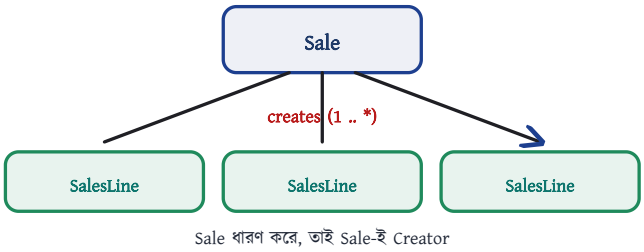
- লাইন আইটেমগুলো কেবল **Invoice**-এর কাছে, তাই যোগফল করতে পারে কেবল সেই।
- কোনো প্যারামিটার পাঠাতে হয় না, কোনো *LineItem*-এর ব্যাক-পয়েন্টার লাগে না।
- জ্ঞান থাকে **যে ক্লাসের কাছে ডেটা, তার ভেতরেই**: কাপলিং কমে, কোহেশন বাড়ে।

পরীক্ষার ফাঁদ: Information Expert মানে "সবচেয়ে বেশি মেথড থাকা ক্লাস" **নয়**। এটি সেই ক্লাস যার কাছে **প্রয়োজনীয় ডেটা** আছে। ডেটা অন্যত্র থাকলে দায়িত্ব সেখানেই যাবে।

২ক ক্রিয়েটর (Creator) — উদাহরণসহ

Creator: ক্লাস **B**-কে ক্লাস **A**-এর ইনস্ট্যান্স তৈরির দায়িত্ব দিন, যদি **যেকোনো একটি** শর্ত সত্য হয়: B **ধারণ** করে A · B **একত্রিত** করে A · B-তে A **রেকর্ড** আছে · B **ঘনিষ্ঠভাবে ব্যবহার** করে A · B-এর কাছে A-কে **initialise** করার ডেটা আছে।

```
class Sale {
    List<SalesLineItem> lines;
    SalesLineItem addLine(Product p, int q) {
        SalesLineItem l = new SalesLineItem(p, q);
        lines.add(l);
        return l;
    }
}
```



Sale ধারণ করে, তাই Sale-ই Creator

ক্রিয়েটর: যে ধারণ করে, সেই তৈরিই করে

মনে রাখার কৌশল: Creator = "C for Container"। B যদি A ধারণ না করেও A-এর প্রতিটি ফিল্ডের initial মান বলতে পারে, তবে সেও স্বাভাবিক Creator — কারণ ওখানে তৈরি করলে কলকারকে যে আর্গুমেন্ট দিতে হয় **কমে যায়**।

পরীক্ষার ফাঁদ: Creator মানে "যে ক্লাস ব্যবহার করে" তা নয়। POS সিস্টেমে **Sale** — Register, ProductCatalog বা Payment নয় — **SalesLineItem** তৈরি করে, কারণ Sale সেগুলোকে *ধারণ* করে।

৩ লো কাপলিং ও হাই কোহেশন — উদাহরণসহ

Low Coupling (লো কাপলিং): এমনভাবে দায়িত্ব বরাদ্দ করুন যাতে **অপ্রয়োজনীয় নির্ভরতা সর্বনিম্ন** থাকে। পরীক্ষায় বলার নিয়ম: **দায়িত্ব সেই ক্লাসকে দিন যা অন্য ক্লাসের ওপর সবচেয়ে কম নির্ভরশীল**। নির্ভরতা মাপা হয় কতটা পরিমাণ অন্য অবজেক্টের **স্টেট** জানতে হয় তা দিয়ে।

খারাপ — হাই কাপলিং

```
class Customer {
    String name;
    void saveToDatabase() {
        // opens a JDBC connection,
        // builds SQL, commits a tx
    }
}
```

ক্ষতি: *Customer* এখন DB, SQL, ড্রাইভার ও নেটওয়ার্ক জানে।

ভালো — লো কাপলিং

```
class CustomerRepository {
    void save(Customer c) {
        // all DB logic lives here
    }
}
class Customer {
    String name;  // no DB knowledge
}
```

লাভ: *Customer* পুনর্ব্যবহারযোগ্য, repository বদলানো যায়, দুটি আলাদা ইউনিট টেস্ট করা যায়।

৩ক হাই কোহেশন — উদাহরণসহ

```
// LOW COHESION — the "god class"
class CustomerManager {
    void browseShop()  { }
    void chargeCard()  { }
    void shipOrder()   { }
    void sendEmail()   { }
    void updateDatabase(){ }
}
```

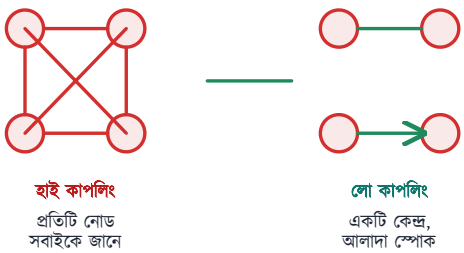
পাঁচটি অসম্পর্কিত কাজ: **পরিবর্তনের কোনো একটি কারণ নেই**, আর প্রতিটি পরিবর্তন পাঁচটিকেই ঝুঁকিতে ফেলে।

```
// HIGH COHESION — split it
class Customer { /* customer data */ }
class Cart     { /* cart operations */ }
class Payment  { /* payment logic  */ }
class Shipping { /* delivery       */ }
class Notifier { /* e-mail         */ }
```

প্রতিটির **পরিবর্তনের একটিমাত্র স্পষ্ট কারণ** আছে — SRP-রই আরেক রূপ।

মনে রাখার কৌশল: **কাপলিং হলো এই বেলুনে কতগুলো সুতো বাঁধা** — চারটি সুতো আর ওড়ে না। **কোহেশন হলো তার নিজের অংশগুলো কতটা জোরে জোড়া লেগে আছে:** পাঁচটি অসম্পর্কিত কাজ = নুড়ির ঢেলি, পাথর নয়।

পরীক্ষার ফাঁদ: দুটি আলাদা মাপকাঠি: কাপলিং হলো ক্লাসগুলোর *মধ্যে*, কোহেশন এক ক্লাসের *ভেতরে*। একটি ক্লাস অত্যন্ত কোহেজিভ হয়েও তিনটি অপ্রয়োজনীয় ক্লাসের সঙ্গে কাপলড হতে পারে।



একই চারটি অবজেক্ট, দুইভাবে যুক্ত

IT link: ব্যাংক কোর তার লেজার নিয়মগুলো ইন্টারফেসের পেছনে রাখে: ব্যবসায়িক ক্লাসগুলো না ছুঁয়েই SQL বদলানো যায়।

৪ পলিমরফিজম, কন্ট্রোলার, ইন্টারমিডিয়ারি ও বাকিগুলো

Polymorphism (পলিমরফিজম): সাবক্লাসকে এমন আচরণ দেবেন না যা সুপারক্লাসে **আগে থেকেই** আছে। দায়িত্বটি সুপারক্লাস বা ইন্টারফেসে রাখুন, রান টাইমে সঠিকটি বেছে নিতে **পলিমরফিজম** দিয়ে হবে। CashPayment · CardPayment · MobileBanking — সবাই *process()* ভাগ করে।

Pure Arbiters (পিওর আর্বাইটার্স): এমন অবজেক্টকে দায়িত্ব দিন যা **দুই বা ততোধিকের মধ্যে সালিস করে** — যে নিয়ম দুপাশের দুজনেরই দরকার কিন্তু কারও নিজের নয়। যে *TaxPolicy* *Invoice* ও *Refund* দুজনই জিজ্ঞাসা করে।

Don't Talk to Strangers: কোনো মেথড অন্যের ওপর অতিরিক্ত নির্ভরতা রাখলে নিয়ম ভাঙে। শুধু **নিজের সরাসরি সহযোগীর** কাছে জিজ্ঞাসা করুন। কখনো *Order*-এর ভেতর দিয়ে *Order.getCustomer().getAddress()* ধরবেন না।

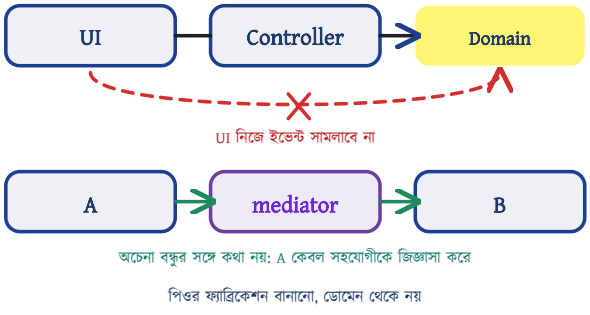
Controller (কন্ট্রোলার): *enterItem*-এর মতো সিস্টেম ইভেন্ট যে অবজেক্ট **UI স্তরের পরে প্রথম** গ্রহণ করে ও সমন্বয় করে। দুই প্রকার: **ফ্যাসেড কন্ট্রোলার** (গোটা সিস্টেমের প্রতিনিধি) ও **ইউজ-কেস / সেশন কন্ট্রোলার** (একটি পরিস্থিতি)। **UI ক্লাসে কখনো নয়।**

Intermediary / Pure Fabrication: এমন ক্লাস যা **কোনো ডোমেন ধারণাই প্রতিনিধিত্ব করে না**, শুধু অস্বাভাবিক নির্ভরতা ভাঙতে বানানো; সাধারণত **নিম্ন-স্তরের সাধারণ প্যাকেজে** রাখা হয়। ইনভয়েস আর ডেটাবেসের মাঝে একটি *ReportService*।

Protected Variations: **অনুমিত পরিবর্তনের** বিন্দুগুলো চিহ্নিত করে প্রতিটির সামনে ইন্টারফেস বা অ্যাবস্ট্রাক্ট ক্লাস রাখুন, যাতে অস্থির অংশ বাকিটা না ছুঁয়েই বদলায়। Open/Closed নীতি; সাধারণত Strategy প্যাটার্ন।

SOLID-এর সঙ্গে নির্দেশিকাগুলোর মিল

গ্র্যাস্প নির্দেশিকা	আসলে এই নীতি
High Cohesion	Single Responsibility নীতি
Low Coupling	Dependency Inversion · Open/Closed
Protected Variations	Open/Closed নীতি · Strategy
Polymorphism	Liskov Substitution
Indirection / Intermediary	Dependency Inversion নীতি



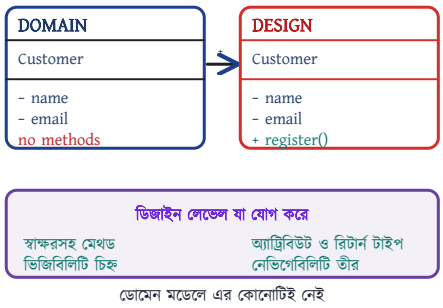
UI-এর পরে কন্ট্রোলার; মিডিয়েটর শর্টকাট কাটে

পরীক্ষার ফাঁদ: Controller নির্দেশিকা মানে "ব্যবহারকারী যে ক্লাসে ক্লিক করেন" **নয়** — ইভেন্ট গ্রহণকারী UI ক্লাস তো গ্র্যাস্প নিষিদ্ধ করে। আর Pure Fabrication ডোমেন মডেল ক্লাস থেকে আলাদা: এর **কোনো ডোমেন অর্থ নেই**, থাকে শুধু decoupling-এর জন্য।

৫ ডিজাইন ক্লাস ডায়াগ্রাম (DCD)

সংজ্ঞা: **ডিজাইন ক্লাস ডায়াগ্রাম** (DCD) হলো **ডিজাইন লেভেলে** আঁকা UML ক্লাস ডায়াগ্রাম: বিশ্লেষণ/ডোমেন মডেলের ক্লাস ডায়াগ্রামের সঙ্গে বাস্তবায়নকারীর দরকারি সবকিছু যোগ করা — স্বাক্ষরসহ **অপারেশন**, **টাইপ**, **ভিজিবিলিটি** চিহ্ন এবং **নেভিগেবিলিটি** তীর।

ডোমেন / বিশ্লেষণ মডেল	ডিজাইন ক্লাস ডায়াগ্রাম
ডোমেনের **ধারণাগত** দৃষ্টিভঙ্গি	বাস্তবায়নের **সফটওয়্যার** দৃষ্টিভঙ্গি
ক্লাস, অ্যাট্রিবিউট, অ্যাসোসিয়েশন, মাল্টিপ্লিসিটি	সেগুলো **মেথড ও রিটার্ন টাইপসহ**
বহুবচন ডোমেন নাম (Sales)	**একবচন** নাম (Sale)
সফটওয়্যার-নির্দিষ্ট টাইপ নেই	সুনির্দিষ্ট টাইপ: String, double, List
ভিজিবিলিটি চিহ্ন নেই	+ public · - private · # protected
অ্যাসোসিয়েশন দিকহীন	**নেভিগেবিলিটি** — রেফারেন্স কে রাখে
ইন্টারফেস বা প্যাকেজ নেই	<<interface>>, <<abstract>>, প্যাকেজ



একই বিষয়, দুইবার: ধারণাগত, তারপর ডিজাইন করা

ডিজাইন থেকে কোডে ম্যাপিং: এক-এক-এক নিয়ম

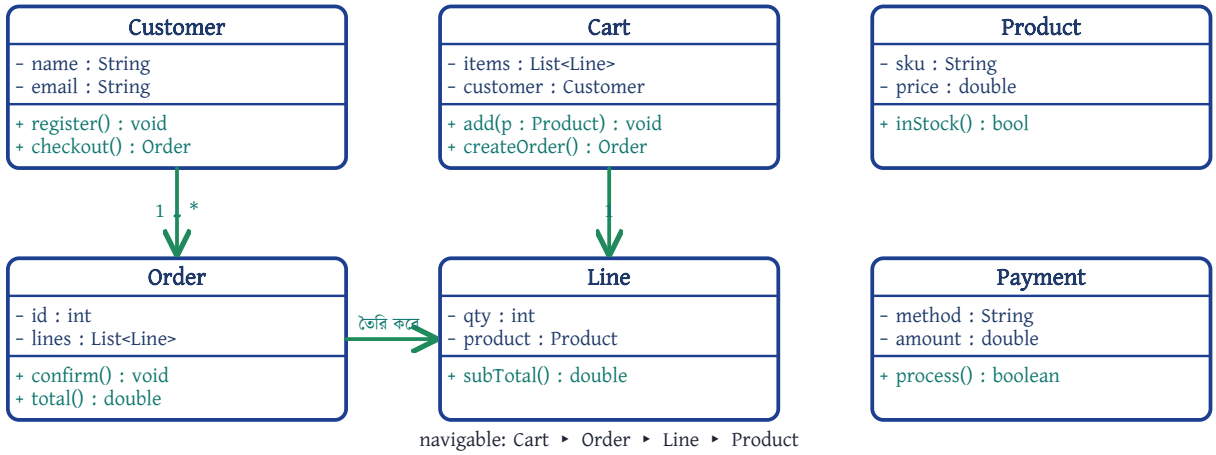
ডিসিডির উপাদান	কোডে পরিণত হয়
একটি ক্লাস বক্স	একটি **ক্লাস** — বা স্টেরিওটাইপ <<interface>> হলে ইন্টারফেস
অ্যাট্রিবিউট – *price : double*	ওই টাইপের একটি **ফিল্ড** / প্রপার্টি
অপারেশন + *total() : double*	ওই রিটার্ন টাইপের একটি **মেথড**
নেভিগেবিলিটিসহ অ্যাসোসিয়েশন 1 → *	একটি রেফারেন্স ফিল্ড ও একটি **কালেকশন** (অ্যারে / *List* / *Set*)
অ্যাগ্রিগেশন ◇ বা কম্পোজিশন ◆	পূর্ণাংশের দ্বারা ধারণ করা whole-part রেফারেন্স
জেনারালাইজেশন △ / রিয়ালাইজেশন ▷	**ইনহেরিট্যান্স** / ডেলিগেশনে কম্পোজিশন

পরীক্ষার ফাঁদ: ডোমেন মডেলে যা নেই কিন্তু ডিসিডিতে আছে, তা হলো একটাই জিনিস: **মেথড (অপারেশন) ও নেভিগেবিলিটি**। কনসেপ্ট ক্লাস, অ্যাসোসিয়েশন ও মাল্টিপ্লিসিটি **দুটোতেই** থাকে।

মনে রাখার কৌশল: ডোমেন মডেল = **কী আছে**। ডিসিডি = **কী আছে, আর সেটি কী করে**। রিটার্ন টাইপসহ মেথড স্বাক্ষর দেখলেই বুঝে যাবেন, এটি ডিসিডি।

ডিসিডি তৈরির চার ধাপ: ① ডোমেন মডেলের প্রতিটি কনসেপ্ট ক্লাস নিন, নাম **একবচনে** করুন (Sales → *Sale*)। ② প্রতিটি ক্লাসের অ্যাট্রিবিউটে **সফটওয়্যার টাইপ** দিন (age: Integer নয়, *int*)। ③ প্রতিটি ক্লাসকে এক-দুইটি করে দায়িত্ব দিয়ে সেগুলোকে অপারেশন হিসেবে সাজান, **রিটার্ন টাইপ** লিখুন। ④ যে অ্যাসোসিয়েশনে রেফারেন্স রাখতে হবে, সেখানে **নেভিগেবিলিটি তীর** আঁকুন — তীর যার দিকে, রেফারেন্স তার কাছে।

৬ একটি ডিসিডি ধাপে ধাপে, আর যে কোড হয়ে দাঁড়ায়



একটি ডিজাইন ক্লাস ডায়াগ্রাম — Customer 1 — * Cart — * Order — * Line — * Product, সঙ্গে একটি Payment সহযোগী; প্রতিটি বাক্সে এখন রিটার্ন টাইপসহ অপারেশন

```
class Customer {
    String name;
    String email;
    Cart cart = new Cart();

    void checkout() {
        Order o = cart.createOrder();
        o.confirm();
    }
}
class Line {
    Product product;   // navigable ▸
    int qty;
    double subTotal() {
        return product.price * qty;
    }
}
```

```
class Cart {
    List<Line> items = new ArrayList<>();

    void add(Product p, int q) {
        items.add(new Line(p, q));
    }
    double calculateTotal() {
        return items.stream()
            .mapToDouble(Line::subTotal)
            .sum();
    }
    Order createOrder() {
        return new Order(items);
    }
}
```

ছবি থেকে ম্যাপিং পড়ুন: *Cart.items : List<Line>* → ফিল্ড *List<Line> items*; *Line.product : Product* → ফিল্ড *Product product*; *Line.subTotal() : double* → মেথড। **ডিসিডির কোথাও কিছু নতুন করে বানাতে হয় না** — ক্লাসের নাম, অ্যাট্রিবিউটের নাম, টাইপ ও মেথডের নামই প্রোগ্রামারের প্রয়োজনীয় শনাক্তকারী।

পরীক্ষার ফাঁদ: ডিসিডি CRC কার্ড থেকে আলাদা জিনিস: CRC দিয়ে দায়িত্ব *আবিষ্কার* করা হয়, ডিসিডিতে তা *লিখে রাখা* হয়। "ডিজাইন সরাসরি কোডে যায়" মানে **এক-এক-এক ম্যাপিং** — প্রতিটি ডায়াগ্রাম উপাদানের আলাদা ফাইল নয়।

৭ স্ট্রাকচারাল বনাম ফাংশনাল টেস্টিং, ইপি ও বিভিএ

বিষয়	হোয়াইট বক্স (স্ট্রাকচারাল, গ্লাস-বক্স)	ব্ল্যাক বক্স (ফাংশনাল, বিহেভিয়ারাল)
টেস্ট কেস কীস থেকে	কোডের **অভ্যন্তরীণ কাঠামো** থেকে	**স্পেসিফিকেশন** থেকে, কোড থেকে নয়
টেস্টারের জ্ঞান	প্রোগ্রামের ভেতরের বিষয় জানেন	কীভাবে লেখা হয়েছে তা **জানেন না**
আরেক নাম	স্ট্রাকচারাল, ক্লিয়ার-বক্স, গ্লাস-বক্স	ফাংশনাল, স্পেসিফিকেশন-ভিত্তিক
মূল প্রশ্ন	"প্রতিটি স্টেটমেন্ট / ব্রাঞ্চ **চলেছে** কি?"	"প্রতিটি বৈধ ইনপুটে **আউটপুট** ঠিক আছে কি?"
পদ্ধতি	বেসিস পাথ টেস্টিং, কভারেজ বিশ্লেষণ	সমতুল্য বিভাজন, বিভিএ, কজ-ইফেক্ট গ্রাফিং
সবচেয়ে ভালো ধরে	ডেড কোড, চালানো হয়নি এমন ব্রাঞ্চ	বাদ পড়া প্রয়োজন, ভুল আউটপুট

টেস্ট বেসিস: যে নথি থেকে টেস্ট বের করা হয়: **হোয়াইট বক্স** টেস্টের বেসিস **ডিজাইন মডেল / কোড**, **ব্ল্যাক বক্স** টেস্টের বেসিস **প্রয়োজনীয়তার স্পেসিফিকেশন**। টেস্টিং ক্রুটের **উপস্থিতি** দেখাতে পারে, অনুপস্থিতি কখনো নয়।

পরীক্ষার ফাঁদ: টেস্টিং প্রোগ্রাম **সঠিক প্রমাণ করতে পারে না**; শুধু আস্থা বাড়ায়। যে টেস্ট **কখনো ব্যর্থ হয়নি**, তাকে ভালো টেস্ট বলা এখনো প্রমাণিত নয়। ইউনিট → ইন্টিগ্রেশন → ভ্যালিডেশন → সিস্টেম লেভেল, ডিবাগিং, রক্ষণাবেক্ষণ ও টেস্ট মেট্রিক্স **টেস্টিং কৌশল, রক্ষণাবেক্ষণ, মান ও মেট্রিক্স** নোটে আছে।

৭ক সমতুল্য বিভাজন ও সীমানা মান বিশ্লেষণ

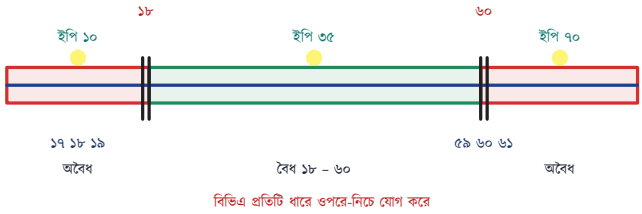
সমতুল্য শ্রেণি (বয়স)	ধরন	ইপি বাছাই
বয়স < ১৮	অবৈধ	১০
১৮ ≤ বয়স ≤ ৬০	**বৈধ**	৩৫
বয়স > ৬০	অবৈধ	৭০

সমতুল্য বিভাজন (EP) ইনপুট ডোমেনকে এমন শ্রেণিতে ভাগ করে যেগুলো একইভাবে ব্যবস্থিত হয়, তারপর প্রতিটি থেকে **একটি প্রতিনিধি** পরীক্ষা করে — কম টেস্ট, বড় কভারেজ। বৈধ ও অবৈধ শ্রেণি **আলাদা করে** গোনা হয়।

সীমানা	বিভিএ বাছাই
নিম্ন, ১৮	১৭ · **১৮** · ১৯
উচ্চ, ৬০	৫৯ · **৬০** · ৬১

বিভিএ (BVA) প্রতিটি সীমানার **ঠিক নিচে, ঠিক ওপরে ও ঠিক মাঝে** পরীক্ষা করে। অফ-বাই-ওয়ান ত্রুটি থাকে **ধারের কাছে** — ঠিক যেখানে ইপি মাত্র একটি নমুনা নেয়।

কখন কোনটি ব্যবহার করবেন ✍️: ইনপুট ডোমেন ছোট হলে (যেমন ১–২০) শুধু ইপিই যথেষ্ট — সব মান না দেখে শ্রেণিগুলোর প্রতিনিধি একটি করে দিলেই ক্লাসে পৌঁছে যায়। ডোমেন বড় হলে (বছর, পরিমাণ, তারিখ) একাই ইপি ঝুঁকি বাড়ায়, তখন **বিভিএ** যোগ করুন। বাস্তবে বেশিরভাগ দলীকৃত চেকলিস্টে **ইপি + বিভিএ** — দুটো একসঙ্গে। কারণ ইপি ধরে রাখে *প্রতিনিধি* আর বিভিএ ধরে রাখে *প্রান্তিক মান*: একটি শ্রেণির ভেতরের মান ভুল হলে ইপি ধরতে পারে না, কারণ সে মাঝের মানের কাছাকাছি যায়।



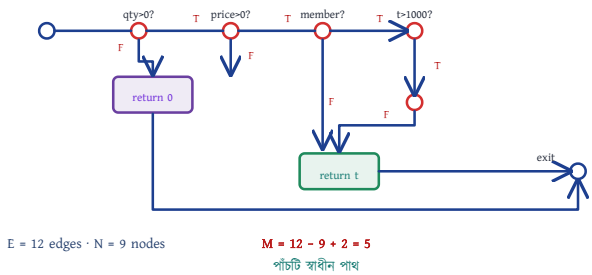
তিনটি সমতুল্য শ্রেণি; বিভিএ দুই ধারে ছয়টি মান যোগ করে

৮ কভারেজ, সাইক্লোম্যাটিক জটিলতা ও বেসিস-পাথ টেস্টিং

কভারেজ মানদণ্ড	কীভাবে মাপা হয়	শক্তি
স্টেটমেন্ট	চলা ÷ মোট এক্সিকিউটেবল স্টেটমেন্ট × ১০০ %	সবচেয়ে দুর্বল — ব্রাঞ্চ ফাঁকা থাকে
ব্রাঞ্চ / ডিসিশন	নেওয়া আউটকাম ÷ মোট আউটকাম × ১০০ %	**সাধারণ ন্যূনতম লক্ষ্য**
কন্ডিশন	মূল্যায়িত শর্ত ÷ মোট শর্ত × ১০০ %	*a && b*-এর প্রতিটি অংশ দুইভাবে
মাল্টিপল কন্ডিশন	n শর্তের সব সত্য-মিথ্যা সংমিশ্রণ — 2^n	খুব শক্তিশালী, খুব ব্যয়বহুল
পাথ	চলা পাথ ÷ মোট পাথ × ১০০ %	সবচেয়ে শক্তিশালী, প্রচেষ্টা সূচকীয়

সাইক্লোম্যাটিক জটিলতা (McCabe, ১৯৭৬): $M = E - N + 2P$ — E = কন্ট্রোল-ফ্লো গ্রাফের শিখা, N = নোড, P = সংযুক্ত অংশ। সাধারণ মডিউলে P = 1, তাই পরিচিত রূপ $M = E - N + 2$। M আসলে স্বাধীন পাথের সংখ্যাই — ফ্লো গ্রাফের ক্ষেত্রসমূহ — এবং সরল শর্তীয় কোডে **M = (স্বাধীন শর্ত) + 1**। কোনো শর্তই না থাকলে M = 1।

```
double total(int qty, double price,
             boolean member) {
  if (qty > 0 && price > 0) {
    double t = qty * price;
    if (member && t > 1000)
      t = t * 0.90;
    return t;
  }
  return 0.0;
}
```



কন্ট্রোল-ফ্লো গ্রাফ: E = ১২, N = ৯, তাই পাঁচটি স্বাধীন পাথ

১. ফ্লো গ্রাফ আঁকুন
২. M গণনা করুন
৩. Mটি স্বাধীন পাথ খুঁজুন
৪. প্রতি পাথে একটি টেস্ট কেস

উদাহরণ ✍️: চারটি মূল শর্তে **M = 4 + 1 = 5**, যা মেলে **E − N + 2 = 12 − 9 + 2 = 5**-এর সঙ্গে। বেসিস সেট: P1 *qty=0 → 0.0* · P2 *price=0 → 0.0* · P3 *2×100*, সদস্য নয় → *200* · P4 *2×100*, সদস্য, *t≤1000 → 200* · P5 *20×100*, সদস্য → *2000×0.9 = 1800*।

স্বাধীন পাথ: এমন পাথ যা অন্তত একটি **নতুন নোড** বা **নতুন শর্ত** আনে, যা আগের কোনো পাথে ছিল না। সংখ্যাটি সবসময় **M**-এর সমান।

ভি-মডেলে ডিজাইন: ব্যবসায়িক প্রয়োজন ↔ **গ্রহণযোগ্যতা** · সিস্টেম প্রয়োজন ↔ **সিস্টেম টেস্ট** · **আর্কিটেকচার** ↔ **ইন্টিগ্রেশন** · **বিস্তারিত ডিজাইন** ↔ **ইউনিট টেস্ট**।

সম্পর্ক ও সীমা: স্টেটমেন্ট ⊂ ব্রাঞ্চ ⊂ কন্ডিশন ⊂ পাথ; McCabe-এর ঐতিহাসিক সীমা **10**।

বিশেষ পরিবেশের টেস্ট লক্ষ্য: রিয়েল-টাইম → টাইমিং, স্ট্রেস · ডিস্ট্রিবিউটেড → লেটেন্সি, আংশিক ব্যর্থতা · ওয়েব → SQL ইনজেকশন, XSS · মোবাইল → ডিভাইস, স্ক্রিন · এমবেডেড → CPU, মেমরি। পূর্ণ পাথ কভারেজ প্রচেষ্টা সূচকীয় বেড়ে যায়, তাই "১০০ % পাথ কভারেজ" কখনোই বাস্তবসম্মত লক্ষ্য নয়।

পরীক্ষার আগের রাতের পাতা

ফ্যাক্ট শিট:

GRASP = General Responsibility Assignment Software Patterns	Carell ও Fowler, ১৯৯১, IEEE Computer
নির্দেশিকা, প্যাটার্নের তালিকা নয়	জানার দায়িত্ব + করার দায়িত্ব
Info Expert = তথ্য থাকা ক্লাস	Creator = ধারণ · একত্রিত · initialise
Low Coupling = সবচেয়ে কম নির্ভরশীল ক্লাস	High Cohesion = পরিবর্তনের একটি কারণ
Polymorphism = সাবক্লাসে নতুন করে লেখা নয়	Pure Arbiters = দুই বা ততোধিকের মাঝে সালিস
Don't Talk to Strangers = কেবল সহযোগীকে জিজ্ঞাসা	Controller = UI-এর পরের প্রথম non-UI অবজেক্ট
Intermediary / Pure Fabrication = বানানো ক্লাস	Protected Variations = পরিবর্তনের ওপর ইন্টারফেস
High Cohesion ≈ SRP · Protected Var. ≈ OCP	Polymorphism ≈ LSP · Intermediary ≈ DIP
ডোমেন মডেল = ধারণাগত, অপারেশন নেই	ডিসিডি = অপারেশন, টাইপ, নেভিগেবিলিটি
ডিসিডি ↔ কোড এক-এক-এক	হোয়াইট = কাঠামো · ব্ল্যাক = স্পেসিফিকেশন
টেস্টিং ত্রুটের উপস্থিতি দেখায়, অনুপস্থিতি নয়	স্টেটমেন্ট · ব্রাঞ্চ · কন্ডিশন · পাথ
মাল্টিপল-কন্ডিশনে 2^n সংমিশ্রণ লাগে	M = E − N + 2P ; সাধারণ মডিউলে P = 1
M = শর্ত + 1 ; শর্ত না থাকলে M = 1	M = স্বাধীন পাথ · McCabe-এর সীমা 10

MCQ-র জন্য সেরা ১৪টি এক-লাইনের তথ্য

1. গ্র্যাস্প প্রস্তাবক **Carell ও Fowler, ১৯৯১**
2. গ্র্যাস্প **দায়িত্ব বরাদ্দের** নির্দেশিকা
3. Information Expert → **ডেটা থাকা** ক্লাস
4. Creator → যে ক্লাস **ধারণ** করে
5. Low Coupling → **সবচেয়ে কম নির্ভরশীল** ক্লাস
8. Pure Fabrication = **ডোমেন অর্থ নেই**
9. Protected Variations → একটি **ইন্টারফেস**
10. ডিসিডিতে **অপারেশন ও নেভিগেবিলিটি**, ডোমেন মডেলে নয়
11. ডিসিডি **এক-এক-এক** কোডে ম্যাপ হয়
12. হোয়াইট **কাঠামো** থেকে; ব্ল্যাক **স্পেস** থেকে

6. High Cohesion → **একটিমাত্র** স্পষ্ট উদ্দেশ্য
7. Controller → প্রথম **non-UI** অবজেক্ট
13. **M = E − N + 2P** সাইক্লোম্যাটিক সংখ্যা
14. বেসিস পাথে ঠিক **M**টি টেস্ট কেস

সেরা ফাঁদগুলো: গ্র্যাস্প GoF ক্যাটালগ নয় · কন্ট্রোলার UI ক্লাস নয় · ক্রিয়েটর "যে ব্যবহার করে" তা নয় · কাপলিং ক্লাসগুলোর *মধ্যে*, কোহেশন একটির *ভেতরে* · ডোমেন মডেলে অপারেশন নেই · "হোয়াইট বক্স সঠিকতা প্রমাণ করে" ভুল · স্টেটমেন্ট কভারেজ **সবচেয়ে দুর্বল** · M = E − N + 2 কেবল P = 1 হলে · বেসিস পাথ Mটি কেস দেয়, পূর্ণ পাথ কভারেজ **নয়** · বিভিএ ধার পরীক্ষা করে, ইপি শ্রেণি থেকে নমুনা নেয়।

সব কৌশল একসঙ্গে: "তথ্য কার কাছে?" = Information Expert · "C for Container" = Creator · অনেক সুতো = কাপলিং, নুড়ির ঢেলি = কোহেশন · রিটার্ন টাইপ মানেই ডিসিডি · হোয়াইট কোড হেঁটে যায়, ব্ল্যাক স্পেস পড়ে · M = শর্ত + 1, আর M পাথে M কেস · ইপি মাঝ থেকে নমুনা নেয়, বিভিএ ধারে হাঁটে।



কভারেজের সিঁড়ি — সবচেয়ে দুর্বল ও সস্তা উপরে

✓ অনুশীলন MCQ (পরীক্ষার ধাঁচে, ১০ নম্বর)

১. কোন নির্দেশিকা **ডেটা থাকা** ক্লাসটিকে ব্যবহার করে?
ক) Creator খ) Controller
গ) Information Expert ঘ) Low Coupling

২. একটি **Sale** অনেক **SalesLineItem** ধারণ করে। কে তৈরি করবে?
ক) Sale খ) Register
গ) ProductCatalog ঘ) Payment

৩. UI স্তরের পরে সিস্টেম ইভেন্ট সামলায়:
ক) Pure Fabrication খ) Controller
গ) Indirection ঘ) Polymorphism

৪. ডিসিডিতে আছে কিন্তু ডোমেন মডেলে নেই:
ক) কনসেপ্ট ক্লাস খ) অ্যাসোসিয়েশন
গ) অপারেশন, নেভিগেবিলিটি ঘ) মাল্টিপ্লিসিটি

৫. গ্র্যাস্প কারা তৈরি করেছেন?
ক) Booch ও Rumbaugh খ) Carell ও Fowler
গ) Beck ও Cunningham ঘ) IEEE

৬. কাপলিং কী নির্ভরতা মাপে?
ক) এক ক্লাসের ভেতরে খ) ক্লাসগুলোর মধ্যে
গ) দুই ধাপের মধ্যে ঘ) হার্ডওয়্যারের ওপর

৭. কোনো শর্ত না থাকলে সাইক্লোম্যাটিক সংখ্যা:
ক) 0 খ) 1
গ) 2 ঘ) -1

৮. E = 14, N = 11, P = 1 ⇒ M =
ক) 3 খ) 4
গ) 5 ঘ) 23

৯. M = 7 হলে বেসিস পাথ কেস:
ক) 6 খ) 7
গ) 8 ঘ) 14

১০. ব্ল্যাক বক্স কেস আসে কোথা থেকে?
ক) সোর্স কোড খ) ফ্লো গ্রাফ
গ) স্পেসিফিকেশন ঘ) CRC কার্ড

উত্তরমালা → ১-গ · ২-ক · ৩-খ · ৪-গ · ৫-খ · ৬-খ · ৭-খ · ৮-গ · ৯-খ · ১০-গ

লিখিত প্রশ্ন ১: গ্র্যাস্পের সংজ্ঞা দাও, প্রস্তাবক ও সাল উল্লেখ করো, এবং **Information Expert** ও **Creator** একটি করে উদাহরণসহ ব্যাখ্যা করো। ১০ নম্বর

১ **গ্র্যাস্প** = General Responsibility Assignment Software Patterns — দায়িত্ব বরাদ্দের নির্দেশিকা ২

২ **প্রস্তাবক** — জেমস ক্যারেল ও মার্টিন ফাউলার, ১৯৯১, *Toward Unified Design Methodologies*, IEEE Computer ২

৩ **Information Expert** — যে ক্লাসের কাছে তথ্য আছে তারই দায়িত্ব; *Invoice.getTotal()*, কারণ Invoice-ই লাইন আইটেম ধারণ করে ৩

৪ **Creator** — যে ক্লাস অন্যকে ধারণ, একত্রিত করে, রেকর্ড রাখে, ঘনিষ্ঠভাবে ব্যবহার করে বা initialise করার ডেটা রাখে, সে তৈরি করে; *Sale* তৈরি করে *SalesLineItem* ৩

condition	y	n	action
	Y	Y	accept
	Y	N	reject
	N	Y	reject
	N	N	reject

২ শর্ত → 2^2 = ৪ সারি

কজ-ইফেক্ট গ্রাফিং থেকে ডিসিশন টেবিল

লিখিত প্রশ্ন ২: একটি মেথডে দুটি নেস্টেড *if* আছে এবং দুটি *&&* শর্ত আছে। এর সাইক্লোম্যাটিক সংখ্যা বের করো এবং কতগুলো বেসিস পাথ টেস্ট কেস লাগবে তা বলো। ৮ নম্বর **উত্তর:** চারটি মূল শর্ত ⇒ **M = 4 + 1 = 5**; **M = E − N + 2P** দিয়ে যাচাই করে ঠিক **পাঁচটি** বেসিস পাথ কেস লিখুন।

পরীক্ষার আগে যাচাই ✓

- ✓ গ্র্যাস্পের প্রস্তাবক ও সাল বলো
- ✓ নয়টি নির্দেশিকা এক লাইনে বলো
- ✓ একটি পরীক্ষায় ডোমেন মডেল ও ডিসিডি আলাদা করো
- ✓ E, N, P থেকে ও শর্ত থেকে M বের করো
- ✓ সংখ্যা দিয়ে ইপি বনাম বিভিএ আলাদা করো

পাশের লেসন: ইউজ কেস মডেল ও ভি-মডেল **ওয়াটারফল ও UML**-এ; টেস্ট লেভেল, ডিবাগিং ও মেট্রিক্স **টেস্টিং কৌশল**-এ।